# Adityaa Saha

B.Tech, Computer Science and Engineering
Indian Institute of Technology Bhubaneswar

23cs01002@iitbbs.ac.in
LinkedIn | GitHub: Adityaa-Saha17
Codeforces: ats17 | LeetCode: whocares17


## Education

| | | | |
|---|---|---|---|
| **B.Tech, Computer Science and Engineering** | Indian Institute of Technology Bhubaneswar | 2023 – 2027 | CGPA: 6.47 |
| Class XII | Shiv Jyoti Convent School | 2023 | 82% |
| Class X | North Point Senior Secondary Public School | 2021 | 90% |

## Experience

**IIT Bhubaneswar, School of Electrical Sciences** *May 2026 – Jul 2026*
*Research Intern | Advisor: Dr. Devashree Tripathy* Certificate | Code

- Optimized architecture-aware **Sparse Matrix Multiplication (SpMM)** kernels in **CUDA** on **NVIDIA T4** GPUs, restructuring global, shared and distributed memory access patterns
- Re-architected the Tensor Core path from **2:4 structured sparsity** to **block-sparse (BCSR)** after identifying that Turing (sm_75) lacks the Ampere-only `mma.sp.sync` instruction
- Scaled execution to multiple GPUs through framework-level partitioning with **PyTorch/NCCL** on dual-T4 nodes, adapting the DSM-style design to a toolchain without `nvcc`
- Built a **CUDA benchmarking harness** comparing memory optimization strategies across shared and global memory

## Projects

**Userspace TCP/IP Stack** *C++23, Linux* GitHub

- Implemented Ethernet, ARP, IPv4/ICMP, UDP and an **RFC 9293 TCP state machine** over a raw **TAP device** via **epoll**
- Handled three-way handshake, four-way close, retransmission with **Karn/Jacobson RTT estimation** and **Reno congestion control** (slow start, congestion avoidance, fast retransmit/recovery)
- Used `std::expected` error handling, bounds-checked cursors over `std::span` and **zero-copy** fixed buffers on the hot path; validated with an HTTP/1.0 server tested via `tcpdump`, `arping` and `curl`

**L4/L7 HTTP Load Balancer** *C++, Linux* GitHub

- Built a load balancer on **edge-triggered, non-blocking epoll** I/O designed for C10K-scale concurrency, with weighted round robin, least connections, **power-of-two-choices** and **consistent hashing** with virtual nodes
- Added active/passive **health checks** with hysteresis and slow-start recovery, plus a **circuit breaker** (closed/open/half-open) to isolate failing backends
- Wrote a state-machine **HTTP/1.1 parser** with keep-alive pooling and **request-smuggling (CL.TE) defenses**; hot-reloads TOML config via atomic pointer swap and exports Prometheus RED metrics

**Agent-Native Ticket Booking Platform** *Go, MySQL, Redis, MCP*

- Built a booking backend in **Go** that exposes ticketing to LLM agents over the **Model Context Protocol**, behind a custom load balancer, circuit breaker and **per-agent rate limiting**; demoed with Claude Desktop, Gemini and Ollama
- Modeled bookings as an **event-sourced state machine** with **idempotency keys**, a **transactional outbox** and **optimistic locking** to prevent double-booking under concurrent agents

**Key–Value Storage Engine (LSM Tree)** *C++* GitHub

- Built an LSM-tree store with a **skip-list memtable**, crash-safe **WAL** and **SSTables** with block indexes and **Bloom filters**
- Designed compaction behind a pluggable strategy interface (**leveled** and **size-tiered**) to compare write vs. read amplification; background flush/compaction uses **atomic manifest swaps** so readers get consistent, non-blocking snapshots

**Recursive DNS Resolver** *C++* GitHub

- Built a UDP/TCP DNS server on a **thread pool** with a hand-rolled **wire codec** (name compression, EDNS0, TC-bit)
- Implemented an **authoritative zone engine** (A, AAAA, CNAME, MX, NS, TXT, SOA) with wildcards, CNAME chasing, negative answers and a conditional forwarder
- Added a **shared-mutex LRU cache** with TTL decay and negative caching, plus per-IP **token-bucket rate limiting**

**32-bit RISC-V CPU Simulator** *C++* GitHub

- Built a cycle-accurate **RV32I** simulator with a **5-stage pipeline** and a single-pass R/I/S/B/J assembler
- Implemented **EX/MEM and MEM/WB forwarding**, load-use interlock stalls and branch flushing on taken branches, with per-cycle stage and register-file tracing

## Technical Skills & Coursework

| | |
|---|---|
| **Languages** | C, C++ (11/17/23), Go, CUDA, Bash |
| **Systems & Networking** | Linux systems programming, epoll and event-driven I/O, sockets, TCP/IP, DNS, HTTP/1.1, multithreading, atomics and lock-free concurrency, storage engines, GPU memory hierarchies |
| **Backend & Tools** | MySQL, Redis, MCP, PyTorch/NCCL, Git, CMake, Prometheus, tcpdump, dig |
| **Coursework** | Data Structures, Design & Analysis of Algorithms, Advanced Algorithms, Operating Systems, Computer Networks, Database Systems, Computer Organization & Architecture, Networks & System Security, Artificial Intelligence |